Section E. Alternative heat and cold wave definitions

Setup

```
packages <- c("readxl", "dlnm", "splines", "forecast", "Metrics", "ISOweek", "dplyr")
invisible(lapply(packages, library, character.only = TRUE))

find_project_root <- function(start = getwd()) {
  path <- normalizePath(start, mustWork = FALSE)
  for (i in seq_len(8)) {
    if (dir.exists(file.path(path, "Code")) && dir.exists(file.path(path, "Data"))) return(path)
    parent <- dirname(path)
    if (identical(parent, path)) break
    path <- parent
  }
  stop("Set LC_DLNM_BASE_DIR to the project root.")
}
base_dir <- Sys.getenv("LC_DLNM_BASE_DIR")
if (!nzchar(base_dir)) base_dir <- find_project_root()
data_dir <- file.path(base_dir, "Data/Combined_data")
daily_dir <- file.path(base_dir, "Data/UTCI_data/Daily_data")
crossbasis_dir <- file.path(base_dir, "Results/Crossbasis_matrix")
function_dir <- file.path(base_dir, "Code/Function")
results_dir <- file.path(base_dir, "Results/percentile-definition-test")
dir.create(results_dir, recursive = TRUE, showWarnings = FALSE)
output_files <- c(
  "table_E5_lc_cv_raw.rds",
  "table_E6_ll_cv_raw.rds",
  "table_E5_single_population.rds",
  "table_E6_multi_population.rds",
  "table_E5_single_population.tex",
  "table_E6_multi_population.tex",
  "percentile_thresholds_by_fold.csv",
  "percentile_thresholds_by_fold.rds"
)
unlink(file.path(results_dir, output_files), force = TRUE)

sets <- 10
size <- 8
forecast_step <- 78
start_ind <- 102
lag_days <- 21
var_df <- 3
lag_df <- 3
degree <- 3
tol <- 1e-2
max_iter_ll <- 20
max_iter_lc <- c(Attiki = 20, Lisbon = 20, Roma = 15)
```

```
region_names <- c("Attiki", "Lisbon", "Roma")
display_region <- c(Attiki = "Athens", Lisbon = "Lisbon", Roma = "Rome")
age_names <- c("Y20_64", "Y65_74", "Y75_84", "Y_GE85")
age_labels <- c("20--64", "65--74", "75--84", "85+")
death_cols <- c(Attiki = 3, Lisbon = 4, Roma = 6)
exposure_cols <- c(Attiki = 9, Lisbon = 10, Roma = 12)

source(file.path(function_dir, "LC_model.R"))
source(file.path(function_dir, "LL_model.R"))
source(file.path(function_dir, "kt_model.R"))
source(file.path(function_dir, "kappa_fit.R"))
source(file.path(function_dir, "crossbasis_mixed_frequency.R"))
source(file.path(function_dir, "dlnm_proc.R"))
source(file.path(function_dir, "DLNM_LC.R"))
source(file.path(function_dir, "DLNM_LC.forecast.R"))
source(file.path(function_dir, "DLNM_LL.R"))
source(file.path(function_dir, "DLNM_LL.forecast.R"))
region_name <- region_names
```

Data

```
read_combined_data <- function(path) {
  list(
    Y20_64 = read_xlsx(file.path(path, "Y20_64_combined.xlsx")),
    Y65_74 = read_xlsx(file.path(path, "Y65_74_combined.xlsx")),
    Y75_84 = read_xlsx(file.path(path, "Y75_84_combined.xlsx")),
    Y_GE85 = read_xlsx(file.path(path, "Y_GE85_combined.xlsx"))
  )
}

combined <- read_combined_data(data_dir)
week_vec <- combined$Y20_64$Week

build_rate_matrix <- function(region) {
  out <- cbind(
    combined$Y20_64[[death_cols[[region]]]] / combined$Y20_64[[exposure_cols[[region]]]],
    combined$Y65_74[[death_cols[[region]]]] / combined$Y65_74[[exposure_cols[[region]]]],
    combined$Y75_84[[death_cols[[region]]]] / combined$Y75_84[[exposure_cols[[region]]]],
    combined$Y_GE85[[death_cols[[region]]]] / combined$Y_GE85[[exposure_cols[[region]]]]
  )
  colnames(out) <- age_names
  rownames(out) <- week_vec
  out * 52
}

load_utci <- function(region) {
  load_crossbasis_history(base_dir,
                          display_region = display_region[[region]],
                          legacy_region = region)
}

dat_list <- setNames(lapply(region_names, build_rate_matrix), region_names)
```

```r
utci_list <- setNames(lapply(region_names, load_utci), region_names)
```

Wave Definitions

```r
build_original_wave <- function(region) {
  idx <- match(week_vec, combined$Y20_64$Week)
  if (anyNA(idx)) {
    stop("Could not align original wave data with percentile-test weeks.")
  }

  out <- data.frame(
    hot_wave = combined$Y20_64[[paste0(region, "_hot_wave3")]][idx],
    cold_wave = combined$Y20_64[[paste0(region, "_cold_wave3")]][idx]
  )
  colnames(out) <- c(paste0(region, "_hot_wave3"),
                     paste0(region, "_cold_wave3"))
  out
}

weekly_wave_counts <- function(daily_data, regions, thresholds, direction) {
  daily_data <- daily_data[order(daily_data$Date), ]
  out <- data.frame(Week = sort(unique(daily_data$Week)))

  for (region in regions) {
    indicator <- if (direction == "hot") {
      daily_data[[region]] > thresholds[[region]]
    } else {
      daily_data[[region]] < thresholds[[region]]
    }
    wave_day <- as.integer(indicator &
                             dplyr::lag(indicator, 1, default = FALSE) &
                             dplyr::lag(indicator, 2, default = FALSE))
    weekly <- tapply(wave_day, daily_data$Week, sum, na.rm = TRUE)
    out[[region]] <- as.numeric(weekly[match(out$Week, names(weekly))])
  }

  out
}

daily_max <- read_xlsx(file.path(daily_dir, "UTCI_daily_max.xlsx"))
daily_min <- read_xlsx(file.path(daily_dir, "UTCI_daily_min.xlsx"))
daily_max$Week <- ISOweek(daily_max$Date)
daily_min$Week <- ISOweek(daily_min$Date)
daily_max <- daily_max[daily_max$Week %in% week_vec, ]
daily_min <- daily_min[daily_min$Week %in% week_vec, ]

build_percentile_thresholds_for_fold <- function(prob, train_end) {
  train_weeks <- week_vec[seq_len(train_end)]
  daily_max_train <- daily_max[daily_max$Week %in% train_weeks, ]
  daily_min_train <- daily_min[daily_min$Week %in% train_weeks, ]

  hot_thresholds <- vapply(region_names, function(region) {
    unname(quantile(daily_max_train[[region]], probs = 1 - prob, na.rm = TRUE))
```

```
  }, numeric(1))
  cold_thresholds <- vapply(region_names, function(region) {
    unname(quantile(daily_min_train[[region]], probs = prob, na.rm = TRUE))
  }, numeric(1))

  list(hot = hot_thresholds, cold = cold_thresholds)
}

build_percentile_wave_for_fold <- function(prob, train_end, test_end) {
  fold_weeks <- week_vec[seq_len(test_end)]
  daily_max_fold <- daily_max[daily_max$Week %in% fold_weeks, ]
  daily_min_fold <- daily_min[daily_min$Week %in% fold_weeks, ]
  thresholds <- build_percentile_thresholds_for_fold(prob, train_end)

  weekly_hot <- weekly_wave_counts(daily_max_fold, region_names, thresholds$hot, "hot")
  weekly_cold <- weekly_wave_counts(daily_min_fold, region_names, thresholds$cold, "cold")

  setNames(lapply(region_names, function(region) {
    out <- data.frame(
      hot_wave = weekly_hot[[region]][match(fold_weeks, weekly_hot$Week)],
      cold_wave = weekly_cold[[region]][match(fold_weeks, weekly_cold$Week)]
    )
    out[is.na(out)] <- 0
    colnames(out) <- c(paste0(region, "_hot_wave3"),
                       paste0(region, "_cold_wave3"))
    out
  }), region_names)
}

wave_specs <- list(
  Null = list(type = "null"),
  Original = list(
    type = "fixed",
    data = setNames(lapply(region_names, build_original_wave), region_names)
  ),
  P2_5 = list(type = "percentile", prob = 0.025),
  P5 = list(type = "percentile", prob = 0.05),
  P10 = list(type = "percentile", prob = 0.10)
)

build_threshold_audit <- function() {
  percentile_specs <- wave_specs[vapply(wave_specs, function(spec) {
    identical(spec$type, "percentile")
  }, logical(1))]

  do.call(rbind, lapply(names(percentile_specs), function(spec_name) {
    prob <- percentile_specs[[spec_name]]$prob
    do.call(rbind, lapply(0:(sets - 1), function(i) {
      train_end <- start_ind + i * size
      test_start <- train_end + 1
      test_end <- train_end + forecast_step
      thresholds <- build_percentile_thresholds_for_fold(prob, train_end)
```

```
      do.call(rbind, lapply(region_names, function(region) {
        data.frame(
          specification = spec_name,
          percentile = prob,
          fold = i + 1,
          train_end_index = train_end,
          test_start_index = test_start,
          test_end_index = test_end,
          train_end_week = week_vec[train_end],
          test_start_week = week_vec[test_start],
          test_end_week = week_vec[test_end],
          region = display_region[[region]],
          hot_threshold = thresholds$hot[[region]],
          cold_threshold = thresholds$cold[[region]],
          stringsAsFactors = FALSE
        )
      }))
    }))
  }))
}

threshold_audit <- build_threshold_audit()
saveRDS(threshold_audit, file.path(results_dir, "percentile_thresholds_by_fold.rds"))
write.csv(threshold_audit, file.path(results_dir, "percentile_thresholds_by_fold.csv"),
          row.names = FALSE)
```

Cross-Basis Helpers

```
make_crossbasis <- function(utci_train, utci_new = NULL) {
  if (is.null(utci_new)) {
    utci_new <- utci_train
  }
  varknots <- equalknots(utci_train, fun = "ns", df = var_df, degree = degree)
  lagknots <- logknots(lag_days, lag_df)
  crossbasis(
    utci_new,
    lag = lag_days,
    argvar = list(fun = "ns", knots = varknots),
    arglag = list(knots = lagknots, df = lag_df)
  )
}
```

Cross-Validation

```
make_wave_for_fold <- function(wave_spec, train_end, test_end) {
  if (identical(wave_spec$type, "null")) {
    return(setNames(vector("list", length(region_names)), region_names))
  }

  if (identical(wave_spec$type, "fixed")) {
    return(lapply(wave_spec$data, function(wave_data) {
      wave_data[seq_len(test_end), , drop = FALSE]
```

```
    }))
  }

  if (identical(wave_spec$type, "percentile")) {
    return(build_percentile_wave_for_fold(wave_spec$prob, train_end, test_end))
  }

  stop("Unknown wave specification type: ", wave_spec$type)
}

make_ll_crossbasis_pair <- function(utci_train, utci_test) {
  # Keep DLNM--LL comparable with the main model-comparison table, which uses
  # create_train_test_sets() for cross-basis construction.
  varknots_train <- equalknots(utci_train, fun = "ns", df = var_df, degree = degree)
  lagknots_train <- logknots(lag_days, lag_df)
  cb_train <- crossbasis(
    utci_train,
    lag = lag_days,
    argvar = list(fun = "ns", knots = varknots_train, df = var_df),
    arglag = list(knots = lagknots_train, df = lag_df)
  )

  varknots_test <- equalknots(cb_train, fun = "ns", df = var_df, degree = degree)
  lagknots_test <- logknots(lag_days, lag_df)
  cb_test <- crossbasis(
    utci_test,
    lag = lag_days,
    argvar = list(fun = "ns", knots = varknots_test, df = var_df),
    arglag = list(knots = lagknots_test, df = lag_df)
  )

  list(train = cb_train, test = cb_test)
}

make_lc_sets <- function(region, wave_spec) {
  lapply(0:(sets - 1), function(i) {
    train_end <- start_ind + i * size
    test_start <- train_end + 1
    test_end <- train_end + forecast_step
    fold_wave <- make_wave_for_fold(wave_spec, train_end, test_end)
    wave_data <- fold_wave[[region]]

    utci_train <- utci_list[[region]][1:train_end, ]
    utci_test <- utci_list[[region]][test_start:test_end, ]

    list(
      train = list(
        dat = dat_list[[region]][1:train_end, ],
        wave = if (is.null(wave_data)) NULL else wave_data[1:train_end, , drop = FALSE],
        cb = make_crossbasis(utci_train)
      ),
      test = list(
        dat = dat_list[[region]][test_start:test_end, ],
```

```
        wave = if (is.null(wave_data)) NULL else wave_data[test_start:test_end, , drop = FALSE],
        cb = make_crossbasis(utci_train, utci_test)
      )
    )
  })
}

run_lc_cv <- function(wave_spec) {
  setNames(lapply(region_names, function(region) {
    cv_sets <- make_lc_sets(region, wave_spec)
    mae_by_set <- lapply(cv_sets, function(set) {
      model <- DLNM_LC(set$train$dat, set$train$cb, set$train$wave,
                       tol = tol, max_iter = max_iter_lc[[region]],
                       region = region)
      fc <- DLNM_LC.forecast(model, set$test$cb, set$test$wave,
                             kt_model_type = "SARIMA",
                             forecast_step = nrow(set$test$dat))
      baseline_forecast <- fc$baseline_LC.forecast
      model_forecast <- fc$DLNM_LC_final.forecast$model.forecast
      rbind(
        baseline = 100 * sapply(seq_along(age_names), function(j) mae(set$test$dat[, j], baseline_foreca
        model = 100 * sapply(seq_along(age_names), function(j) mae(set$test$dat[, j], model_forecast[, j
      )
    })
    out <- Reduce(`+`, mae_by_set) / length(mae_by_set)
    colnames(out) <- age_labels
    out
  }), region_names)
}

make_ll_sets <- function(wave_spec) {
  lapply(0:(sets - 1), function(i) {
    train_end <- start_ind + i * size
    test_start <- train_end + 1
    test_end <- train_end + forecast_step
    fold_wave <- make_wave_for_fold(wave_spec, train_end, test_end)

    train_dat <- test_dat <- train_wave <- test_wave <- train_cb <- test_cb <- list()
    for (region in region_names) {
      utci_train <- utci_list[[region]][1:train_end, ]
      utci_test <- utci_list[[region]][test_start:test_end, ]
      train_dat[[region]] <- dat_list[[region]][1:train_end, ]
      test_dat[[region]] <- dat_list[[region]][test_start:test_end, ]
      train_wave[region] <- list(if (is.null(fold_wave[[region]])) NULL else fold_wave[[region]][1:train
      test_wave[region] <- list(if (is.null(fold_wave[[region]])) NULL else fold_wave[[region]][test_sta
      cb_pair <- make_ll_crossbasis_pair(utci_train, utci_test)
      train_cb[[region]] <- cb_pair$train
      test_cb[[region]] <- cb_pair$test
    }

    list(train = list(dat = train_dat, wave = train_wave, cb = train_cb),
         test = list(dat = test_dat, wave = test_wave, cb = test_cb))
  })
```

```
}

run_ll_cv <- function(wave_spec) {
  cv_sets <- make_ll_sets(wave_spec)
  mae_by_set <- lapply(cv_sets, function(set) {
    model <- DLNM_LL(set$train$dat, set$train$cb, set$train$wave,
                     tol = tol, max_iter = max_iter_ll)
    fc <- DLNM_LL.forecast(model, set$test$cb, set$test$wave,
                           forecast_step = nrow(set$test$dat[[1]]))
    baseline_forecast <- lapply(fc$baseline_LL.forecast, function(x) exp(as.matrix(x)))
    model_forecast <- lapply(fc$DLNM_LL_final.forecast$model.forecast, function(x) exp(as.matrix(x)))

    setNames(lapply(region_names, function(region) {
      out <- rbind(
        baseline = 100 * sapply(seq_along(age_names), function(j) mae(set$test$dat[[region]][, j], base
        model = 100 * sapply(seq_along(age_names), function(j) mae(set$test$dat[[region]][, j], model_f
      )
      colnames(out) <- age_labels
      out
    }), region_names)
  })

  setNames(lapply(region_names, function(region) {
    Reduce(`+`, lapply(mae_by_set, `[[`, region)) / length(mae_by_set)
  }), region_names)
}

lc_cv <- lapply(wave_specs, run_lc_cv)
ll_cv <- lapply(wave_specs, run_ll_cv)
saveRDS(lc_cv, file.path(results_dir, "table_E5_lc_cv_raw.rds"))
saveRDS(ll_cv, file.path(results_dir, "table_E6_ll_cv_raw.rds"))
```

Tables

```
build_output_matrix <- function(cv_result, baseline_label, model_labels) {
  setNames(lapply(region_names, function(region) {
    out <- data.frame(
      Age = age_labels,
      Baseline = cv_result$Null[[region]]["baseline", ],
      Null = cv_result$Null[[region]]["model", ],
      Original = cv_result$Original[[region]]["model", ],
      P2_5 = cv_result$P2_5[[region]]["model", ],
      P5 = cv_result$P5[[region]]["model", ],
      P10 = cv_result$P10[[region]]["model", ],
      check.names = FALSE
    )
    names(out) <- c("Age", baseline_label, model_labels)
    out
  }), region_names)
}

table_E5 <- build_output_matrix(
  lc_cv,
```

```
  "LC",
  c("DLNM--LC (Null)", "DLNM--LC", "DLNM--LC (2.5\\%)",
    "DLNM--LC (5\\%)", "DLNM--LC (10\\%)")
)

table_E6 <- build_output_matrix(
  ll_cv,
  "LL",
  c("DLNM--LL (Null)", "DLNM--LL", "DLNM--LL (2.5\\%)",
    "DLNM--LL (5\\%)", "DLNM--LL (10\\%)")
)

format_num <- function(x) formatC(as.numeric(x), format = "f", digits = 4)

format_best_num <- function(x) {
  x <- as.numeric(x)
  formatted <- format_num(x)
  best <- min(x, na.rm = TRUE)
  ifelse(abs(x - best) < sqrt(.Machine$double.eps),
         paste0("\\textbf{", formatted, "}"),
         formatted)
}

make_shortstack <- function(top, bottom = "") {
  if (nzchar(bottom)) {
    return(paste0("\\shortstack{\\small\\textbf{", top,
                  "}\\\\\\small\\textbf{", bottom, "}}"))
  }

  paste0("\\shortstack{\\small\\textbf{", top, "}\\\\\\textbf{}}")
}

make_model_header <- function(label) {
  suffix <- regmatches(label, regexpr("\\s*\\([^)]*\\)$", label))
  if (length(suffix) > 0 && nzchar(suffix)) {
    top <- sub("\\s*\\([^)]*\\)$", "", label)
    bottom <- trimws(suffix)
  } else {
    top <- label
    bottom <- ""
  }
  make_shortstack(top, bottom)
}

make_percentile_latex_table <- function(table_list, caption, label) {
  column_labels <- names(table_list[[1]])[-1]
  lines <- c(
    "\\begin{table}[H]",
    "\\centering",
    "\\scalebox{0.9}{",
    "\\begin{tabular}{ccccccc}",
    "\\toprule"
  )
```

```
  for (region_index in seq_along(region_names)) {
    region <- region_names[[region_index]]
    region_table <- table_list[[region]]
    header <- c(make_shortstack(display_region[[region]]),
                vapply(column_labels, make_model_header, character(1)))

    if (region_index > 1) {
      lines <- c(lines, "\\midrule")
    }

    lines <- c(lines, paste0(paste(header, collapse = " & "), " \\\\"), "\\midrule")

    for (age_index in seq_len(nrow(region_table))) {
      values <- format_best_num(unlist(region_table[age_index, -1], use.names = FALSE))
      lines <- c(
        lines,
        paste0(
          region_table$Age[[age_index]], " & ",
          paste(values, collapse = " & "),
          " \\\\"
        )
      )
    }
  }

  lines <- c(
    lines,
    "\\bottomrule",
    "\\end{tabular}",
    "}",
    paste0("\\caption{\\small ", caption, "}"),
    paste0("\\label{", label, "}"),
    "\\end{table}"
  )

  paste(lines, collapse = "\n")
}

latex_E5 <- make_percentile_latex_table(
  table_E5,
  "MAE from 10-fold expanding-window cross-validation for single-population models.",
  "tab:dlnm_lc_percentile"
)

latex_E6 <- make_percentile_latex_table(
  table_E6,
  "MAE from 10-fold expanding-window cross-validation for multi-population models.",
  "tab:dlnm_ll_percentile"
)

saveRDS(table_E5, file.path(results_dir, "table_E5_single_population.rds"))
saveRDS(table_E6, file.path(results_dir, "table_E6_multi_population.rds"))
writeLines(latex_E5, file.path(results_dir, "table_E5_single_population.tex"))
```

```
writeLines(latex_E6, file.path(results_dir, "table_E6_multi_population.tex"))

cat(latex_E5)
```

Athens	LC	DLNM–LC (Null)	DLNM–LC	DLNM–LC (2.5%)	DLNM–LC (5%)	DLNM–LC (10%)
20–64	0.0258	0.0237	**0.0231**	0.0241	0.0239	0.0241
65–74	0.1636	0.1352	0.1355	0.1361	0.1366	**0.1341**
75–84	0.5009	0.3906	0.3675	0.3860	0.3869	**0.3648**
85+	1.8120	1.3086	1.2836	1.3062	1.3283	**1.2597**
Lisbon	**LC**	**DLNM–LC (Null)**	**DLNM–LC**	**DLNM–LC (2.5%)**	**DLNM–LC (5%)**	**DLNM–LC (10%)**
20–64	0.0274	0.0260	**0.0257**	0.0262	0.0268	0.0274
65–74	**0.1500**	0.1599	0.1592	0.1603	0.1582	0.1584
75–84	**0.4615**	0.5251	0.5032	0.5215	0.5200	0.5330
85+	1.8087	1.7477	**1.7293**	1.7744	1.7595	1.7713
Rome	**LC**	**DLNM–LC (Null)**	**DLNM–LC**	**DLNM–LC (2.5%)**	**DLNM–LC (5%)**	**DLNM–LC (10%)**
20–64	0.0197	0.0198	0.0204	0.0195	0.0197	**0.0193**
65–74	**0.1189**	0.1412	0.1491	0.1355	0.1394	0.1403
75–84	**0.3455**	0.3768	0.3511	0.3611	0.3642	0.3754
85+	1.4920	1.4910	1.4388	**1.3391**	1.3795	1.4463

Table 1: MAE from 10-fold expanding-window cross-validation for single-population models.

```
cat("\n\n")
```

```
cat(latex_E6)
```

Athens	LL	DLNM–LL (Null)	DLNM–LL	DLNM–LL (2.5%)	DLNM–LL (5%)	DLNM–LL (10%)
20–64	0.0285	0.0246	0.0233	0.0243	0.0238	**0.0232**
65–74	0.1830	0.1606	0.1565	0.1562	0.1608	**0.1494**
75–84	0.5091	0.4101	0.3767	0.3801	0.3949	**0.3594**
85+	2.0056	1.4937	1.5514	1.3471	1.4345	**1.3097**
Lisbon	**LL**	**DLNM–LL (Null)**	**DLNM–LL**	**DLNM–LL (2.5%)**	**DLNM–LL (5%)**	**DLNM–LL (10%)**
20–64	0.0274	0.0261	**0.0257**	0.0261	0.0266	0.0269
65–74	0.1624	0.1405	**0.1382**	0.1424	0.1411	0.1402
75–84	0.5012	0.4197	**0.4055**	0.4189	0.4065	0.4169
85+	2.0249	1.7761	**1.6425**	1.7330	1.6536	1.7393
Rome	**LL**	**DLNM–LL (Null)**	**DLNM–LL**	**DLNM–LL (2.5%)**	**DLNM–LL (5%)**	**DLNM–LL (10%)**
20–64	0.0192	0.0191	0.0206	**0.0185**	0.0188	0.0185
65–74	0.1276	0.1251	0.1475	**0.1231**	0.1254	0.1272
75–84	0.3927	0.3444	**0.3254**	0.3327	0.3410	0.3452
85+	1.7384	1.4540	1.3179	**1.2673**	1.2859	1.3409

Table 2: MAE from 10-fold expanding-window cross-validation for multi-population models.